

DeepStress: Stress-Testing Deep Search Agents

Ismaël Rousseau^{1,2}, Géraldine Damnati¹, Frédéric Béchet²

¹Orange Research, Lannion, France

²Aix-Marseille Univ., CNRS, LIS UMR 7020, 13000 Marseille, France

{ismael.rousseau, geraldine.damnati}@orange.com
frederic.bechet@lis-lab.fr

Abstract

While search agents demonstrate impressive capabilities in multi-step question answering, their robustness to poor-quality evidence remains under-explored. This phenomenon occurs rarely in realistic benchmarks but can lead to dramatic failure in real life applications. Therefore in this study we propose **DeepStress**, a stress testing framework that controls the frequency of challenging evidence by replacing the retrieval module of search agents with a controlled synthetic environment. We use this framework to control three dimensions that can affect document reliability: trustworthiness, relevance, and factuality. Testing several search agents on HotpotQA and BrowseComp-Plus, we demonstrate that agents exhibit substantial differences in their ability to handle unreliable information and propose new metrics that better document systems outcomes as well as the interactions between conflicting parametric and retrieved knowledge.

1 Introduction

Search is a common way to improve LLM performance on knowledge-intensive tasks [Zhao et al. \(2024\)](#). In Retrieval-Augmented Generation (RAG), a model first retrieves passages relevant to the user query, then conditions its answer on the retrieved evidence. *Search agents* extend this paradigm by making retrieval iterative: they interleave reasoning steps with calls to external search tools, gathering evidence before producing a final answer. Their performance depends not only on the language model used for synthesis, but also on how evidence is retrieved, how reliable it is, and how the agent adapts its search and reasoning strategy to the evidence it receives. However, standard search-agent evaluations often reduce performance to final-answer accuracy, hiding qualitatively different failure modes such as poor retrieval, misleading evidence, or ineffective search behavior.

Recent work, such as BrowseComp-Plus [Chen et al. \(2025\)](#), decouples retriever and model performance by utilizing a curated corpus where questions are paired with supportive and hard-negative documents. However, robustness failures are typically observed *post-hoc*, and the low-signal conditions are difficult to reproduce. Our study explicitly focuses on the *reasoning and search strategies* of these agents. Inspired by *stress-testing* methodologies, we introduce **DeepStress**, a simulation environment designed to evaluate the robustness of deep search strategies. Much like simulated environments used for autonomous vehicles, where extreme weather or traffic conditions are generated to test resilience, **DeepStress** subjects search agents to controlled low-signal evidence conditions. It intercepts the agent’s search calls and replaces the retrieval module with a synthetic environment, allowing us to precisely manipulate the properties of the returned documents. Specifically, we degrade document quality along three orthogonal axes: *trustworthiness*, *relevance*, and *factuality*. Overall, **DeepStress** is not a replacement for real search benchmarks but acts as a controlled laboratory for stress testing of search-agent behavior. Our contributions are as follows:

1. We introduce **DeepStress**, a controlled simulation stress test environment that evaluates search agents by intercepting tool calls and returning synthetic documents modified along three axes: *trustworthiness*, *relevance*, and *factuality*.
2. We evaluate several agents across multiple experimental conditions and document substantial behavior differences when facing degraded documents and propose a Reliability-Aware Score that we evaluate against computational costs.

2 Related Work

Search agents. Early work such as ReAct [Yao et al. \(2022\)](#) and IRCoT [Trivedi et al. \(2023\)](#) showed that off-the-shelf LLMs can be prompted to interleave reasoning with external actions or retrieval, enabling multi-step information seeking without task-specific training. More recent work trains search behavior directly. Search-R1 [Jin et al. \(2025\)](#), for instance, uses reinforcement learning (RL) on multi-hop question-answering (QA) tasks, with rewards derived from final-answer correctness, to encourage models to issue useful search queries during reasoning. ZeroSearch [Sun et al. \(2025\)](#) further removes the dependence on a live search engine during training by using a fine-tuned language model to generate documents that mimic search-engine outputs. A parallel line of work targets *deep-research agents*, which produce longer, citation-rich answers over multiple rounds of retrieval and synthesis. DR-Tulu [Shao et al. \(2025\)](#), for example, uses evolving rubrics as RL rewards to improve long-form research answers. These works focus on improving search-agent capabilities under a given retrieval process. In contrast, our goal is to evaluate how search agents behave under controlled retrieval environment.

Evaluation environments. Search agents are commonly evaluated on their ability to gather relevant evidence through iterative reasoning and search. Multi-hop QA datasets require models to locate and combine evidence about multiple entities before answering. HotpotQA [Yang et al. \(2018\)](#) introduced a diverse set of questions over Wikipedia entries, together with supporting sentence-level facts required for reasoning over the question. More recent benchmarks such as BrowseComp [Wei et al. \(2025\)](#) introduce harder questions answerable through web browsing, where the answer is short and easily verifiable but difficult to find, requiring persistent search over entangled constraints. BrowseComp-Plus [Chen et al. \(2025\)](#) further improves experimental control by using a fixed corpus with human-verified supporting documents and mined challenging negatives, allowing to disentangle the retriever effect from that of the LLM. Our work follows this line of controlled evaluation, but isolates a different factor: how agents react to changes in the quality of retrieved evidence.

Beyond final-answer accuracy. Search agents can fail in many different ways. Even at the outcome level, a binary correct/incorrect score hides important distinctions: an agent may give a wrong answer, express uncertainty, fail to produce a final answer, or exhaust its tool-call budget. Recent work on agent evaluation has therefore argued for more detailed failure analysis, including taxonomies of reasoning, planning, tool-use, and system-execution errors [Deshpande et al. \(2025\)](#). Beyond final outcome, the trajectory itself is also informative. For search agents, failures may come from the way they search, react to evidence, or decide to stop. Prior work has studied search trajectories through query reformulation and evidence reuse [Ning et al. \(2026\)](#), and has shown that additional search calls can be helpful, wasteful, or harmful depending on retrieval conditions [Shi et al. \(2025\)](#); [Xie et al. \(2026\)](#). Our work combines these two views: we report fine-grained final-answer outcomes and analyze whether agents detect and verbalize degradations in retrieved documents.

Robustness to knowledge conflict. By manipulating the quality of retrieved documents, our setting creates controlled cases where agents must reason from evidence that may be unreliable or conflicting, connecting our work to prior studies on knowledge conflict. DRAGged into Conflicts [Cattan et al. \(2025\)](#) introduces a taxonomy of conflict situations in RAG, together with the expected model behavior for each category, and shows that LLMs often struggle to resolve conflicts between retrieved sources. TRACK [Feng et al. \(2026\)](#) studies how LLMs propagate newly provided facts through multi-step reasoning

when these facts conflict with the model’s initial parametric knowledge. Our work differs from these studies in that we evaluate active search agents rather than static RAG or in-context reasoning pipelines. Instead of only asking whether a model can resolve a conflict once the evidence is given, we study how agents behave while searching under controlled degradations of the retrieved documents.

3 Methodology

3.1 Simulation environment

To clearly highlight differences in agent behaviors during search, we replace the standard search tool with a simulated environment where we can precisely control the properties of documents provided to the agent’s context. Documents are dynamically generated for each agent query instead of being precomputed. Agentic search is by nature interactive: different agents, or even the same agent under different conditions, may issue different queries for the same question. Dynamic generation ensures that the simulated search result is tailored to the actual query made by the agent while still preserving the controlled reliability attributes imposed by the scenario. Since documents are generated rather than retrieved from a fixed corpus or the web, we restrict each search call to a single document ($top_k = 1$). Returning one controlled document per call may differ from traditional retrieval settings, but gives us finer control over the evidence shown to the agent and places the experiment in a laboratory-style setting.

Let $\{D_1, D_2, \dots, D_n\}$ be the set of dimensions under study. A scenario γ specifies a probability for each dimension, chosen by the experimenter:

$$\pi_\gamma = (\pi_\gamma(D_1), \dots, \pi_\gamma(D_n)) \quad (1)$$

For each simulated search call, we sample one binary label per dimension from the scenario probabilities. These labels define the target properties of the returned document. In the experiments reported in this paper, they are translated into dimension-specific prompt instructions, although the framework itself is not tied to prompt-based generation.

For example, if D_k corresponds to factuality and $\pi_\gamma(D_k) = 80\%$, then the document is sampled as *factually correct* with probability 80% and *factually incorrect* with probability 20%. In our implementation, this sampled label determines whether the dynamic prompt includes supporting evidence or instructions to introduce factual errors. The value $\pi_\gamma(D_k)$ therefore defines the expected quality for dimension D_k , not the exact realized counts: over 50 search calls, the expected number of factually correct documents is 40, but the realized number may differ because each document is sampled independently.

The code for this environment is publicly available¹ and is sufficiently flexible to allow users to define their own simulation processes.

3.2 Simulating document reliability

In this paper, we focus on a controlled experimental setup where we manipulate three source attributes:

- **Trustworthiness (T)**: whether the source appears credible based on its style, structure and URL, regardless of its factuality.
- **Relevance (R)**: whether the source meaningfully addresses the model-generated query rather than covering unrelated content.
- **Factuality (F)**: whether the claims contained in the source are factually correct.

We generate documents with an LLM, using the sampled values of T , R , and F to control the output. For each search call, the simulator generates one synthetic document with a title and URL; its length is chosen to approximate a Wikipedia paragraph. The agent’s search query is

¹The repository link will be provided after peer-review.

passed to the LLM as the query the document should address, while the sampled labels are converted into prompt instructions. For factually correct documents, we ground generation in dataset-provided evidence: gold Wikipedia paragraphs for HotpotQA and human-verified evidence documents for BrowseComp-Plus. This reduces accidental errors on details such as dates or locations. Full prompt and generation details are given in Appendix D.1.

Factually correct documents ($F=1$) are grounded in the selected support context, while *factually incorrect* documents ($F=0$) are prompted to introduce severe factual errors. For *trustworthy* documents ($T=1$), we generate a plausible reliable URL and page title. For *low-trust* results ($T=0$), we sample a domain from the OpenSources list², generate a plausible page title for that domain, and include specific instructions in the content generation prompt. For *irrelevant* documents ($R=0$), we replace the generation target with an unrelated question sampled from the dataset and use its associated supporting context. To mimic a realistic partial web search, the query passed to the generator is a document title sampled from that unrelated support context. This produces a coherent and grounded document that is useful for another information need, but irrelevant to the original question asked to the agent.

3.3 Sanity check

As our framework heavily relies on synthetically generated documents produced under controlled degradation conditions, we conduct a human evaluation to verify that the intended trustworthiness and factuality attributes are faithfully reflected in the generated documents. Examples of these documents are provided separately in Appendix A.3.

A subset of 50 randomly sampled documents is manually annotated by three annotators. Details of the annotation protocol and full results are provided in Appendix A.1. The evaluation shows high agreement with the intended trustworthiness labels (96.4%), together with strong inter-annotator agreement. Accuracy is lower for factuality (85.5%), with slightly weaker inter-annotator agreement, suggesting that factuality is more difficult to assess. Overall, these results indicate strong alignment between the intended attributes and those perceived by human annotators.

3.4 Metrics

In order to verify how agents react under degraded conditions, we must define metrics that reflect various possible behaviors.

Final-answer outcomes. Let $\mathcal{D}$ be a set of evaluated runs. Each run $i \in \mathcal{D}$ is associated with a final answer a_i and a document-quality condition γ_i . We analyze final answers through five outcomes, going beyond a binary correct/incorrect evaluation:

- **Correct** (C_i): the final answer is factually equivalent to the ground truth.
- **Incorrect** (I_i): the model gives a confident but factually wrong answer.
- **Abstention** (A_i): the model states that it doesn't know or have enough information.
- **Exceed tool budget** (B_i): the agent reaches its tool-call limit before answering.
- **Missing answer tag** (M_i): the run terminates without a closing answer tag (e.g. formatting error or premature `<eos>` token)

Runs that exceed tool budget or miss the answer tag are assigned directly to their corresponding category. For runs that contain a final answer, we use GPT-4.1 as an LLM-as-a-judge to classify the answer as C_i , I_i or A_i . We adapt the original BrowseComp evaluation prompt to include the abstention category. Additional evaluation details are provided in Appendix E.

Tokens Per Correctness (TPC). Xie et al. (2026) define the cost of run i as a function of the number of generated tokens g_i and input tokens x_i , and the sequence of search calls S_i :

$$\text{Cost}_i = g_i + \lambda x_i + \mu |S_i|, \quad (2)$$

²<https://github.com/OpenSourcesGroup/opensources>

where, $\lambda = 0.25$ and $\mu = 500$ weight the relative cost of input tokens and search calls. They introduce TPC, which estimates how much computation cost is spent per correct answer:

$$\text{TPC}(\mathcal{D}) = \frac{\sum_{i \in \mathcal{D}} \text{Cost}_i}{\sum_{i \in \mathcal{D}} C_i}. \quad (3)$$

This score measures efficiency relatively to raw correctness (the lower the more efficient), but it does not distinguish whether the agent failed despite reliable evidence or avoided answering when the simulated evidence was unreliable.

Reliability-Aware Score (RAS). We address this distinction by introducing RAS which incorporates the document-quality condition. In our setting, every question is answerable in principle, but the simulator can make the available evidence unreliable or uninformative. In the extreme case where relevance or factuality is set to 0%, the agent should not be expected to recover the answer from the provided documents. RAS therefore gives full credit to correct answers, no credit to incorrect or malformed answers, and **partial credit** to abstention-like behavior under unreliable conditions.

For a given scenario γ , under conditions $\{\pi_\gamma(T), \pi_\gamma(R), \pi_\gamma(F)\}$ for respectively trustworthiness, relevance, and factuality, we define the reliability of condition γ as the product:

$$\rho_\gamma = \pi_\gamma(T) \times \pi_\gamma(R) \times \pi_\gamma(F). \quad (4)$$

The reliability-aware score (RAS) for run i is:

$$\text{RAS}_i = \begin{cases} 1, & C_i, \\ 1 - \rho_{\gamma_i}, & A_i \vee B_i, \\ 0, & I_i \vee M_i. \end{cases} \quad (5)$$

Thus, correct answers receive full credit. Abstentions and tool-budget exhaustion receive partial credit when reliable evidence is unlikely, while incorrect answers and missing answer tags receive no credit. The aggregate RAS is averaged over runs.

Cost per Reliability-Aware Score. To obtain an analogous cost-normalized metric, we define Cost per Reliability-Aware Score (CoPRAS):

$$\text{CoPRAS}(\mathcal{D}) = \frac{\sum_{i \in \mathcal{D}} \text{Cost}_i}{\sum_{i \in \mathcal{D}} \text{RAS}_i}. \quad (6)$$

If the denominator is zero, CoPRAS is set to $+\infty$. As with any aggregate metrics, RAS and CoPRAS reflect a particular scoring choice; we discuss alternative choices and their impact in Appendix B.

4 Experimental Protocol

4.1 Datasets

We draw questions from two QA datasets that differ in both question difficulty and exposure to LLM pretraining. **HotpotQA** questions typically require a few reasoning hops over Wikipedia passages and are answerable from a small set of supporting paragraphs. **BrowseComp-Plus**, derived from BrowseComp, instead contains queries constructed by inversion from a target entity under multiple constraints; answering them requires iterative search and aggregation across many documents, and state-of-the-art retrievers achieve low recall on its corpus of adversarially mined distractors.

These two datasets are complementary as they differ in their dependency to support documents. HotpotQA answers are often related to general knowledge that can be memorized in the model parameters, besides the corpus itself that is likely to have been memorized during the LLM training. Therefore, by controlling the factuality of the supportive documents we can study how a model deals with contradiction between parametric and dynamic knowledge provided by the search tools. BrowseComp-Plus questions cannot be answered

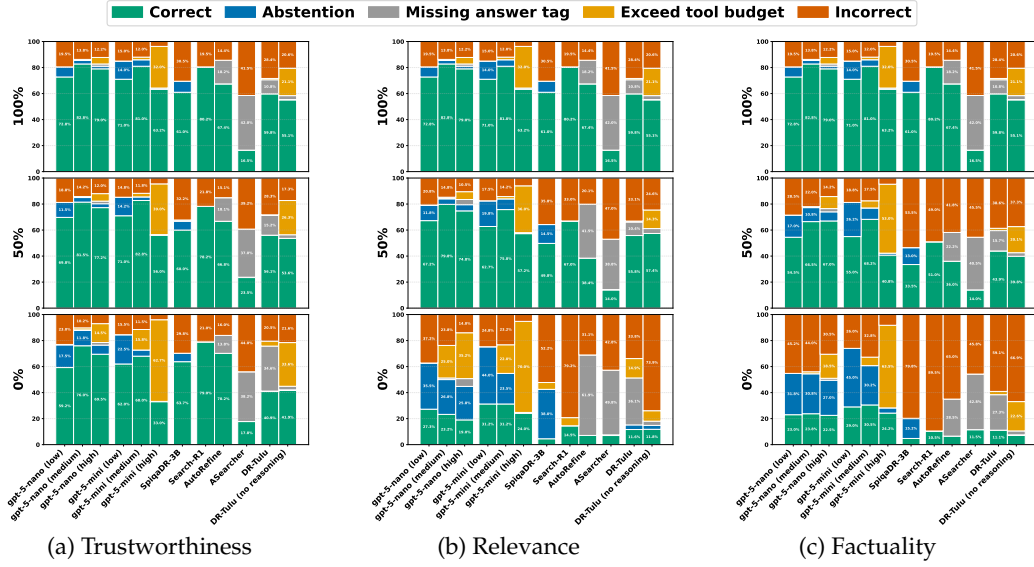


Figure 1: Final-answer outcomes aggregated over HotpotQA and BrowseComp-Plus. Outcome rates across three quality levels (0%, 50%, and 100%) for each document-quality axis: Trustworthiness, Relevance, and Factuality.

without support documents, and the corpus is not memorized by current LLMs, therefore there is no conflict between parametric and dynamic knowledge. This corpus allows us to study how a search agent model can cope with contradictory support documents and how it affects its reasoning strategy.

4.2 Evaluated systems

We have selected systems to cover a diverse set of search-agent designs, including closed-weight and open-weight models, different model sizes, and different training procedures such as supervised tuning, RL under different reward objectives etc. Twelve systems are evaluated. **GPT systems** (gpt-5-nano and gpt-5-mini), each evaluated with varying reasoning efforts (low, medium, and high) serve as 6 closed-weight tool-calling baselines with controllable reasoning effort. **SpigaDR³** is a 3B Qwen2.5-based search agent trained with RL for scientific literature search (see Appendix D.3 for a detailed description of this system trained by our team). **Search-R1** Jin et al. (2025) is a 7B Qwen2.5-based search agent trained with RL to interleave reasoning and multi-turn search. **AutoRefine** Shi et al. (2026) is a 3B Qwen2.5-based retrieval-augmented reasoning model that alternates search with explicit evidence refinement steps before answering. **ASearcher** Gao et al. (2026) is a 7B Qwen2.5-based search agent trained with asynchronous RL for long-horizon information seeking; in our experiments, we use its local-RAG variant rather than the web-search version. **DR-Tulu** Shao et al. (2025) is an 8B open deep-research model trained for long-form, citation-grounded research using RL with evolving rubrics. We also evaluate a **DR-Tulu (no reasoning)** variant, using the same checkpoint but a modified orchestration wrapper. Whenever the model opens a <think> block, the wrapper immediately closes it, preventing explicit reasoning while keeping the same tool-use and answer format.

A system denotes a model together with its harness, *i.e.* the orchestration code and prompts that determine how context is assembled, how tools are invoked, and how model outputs are parsed. To keep the comparison controlled, simulated documents are formatted according to the conventions expected by the corresponding harness. Additional implementation details, including models versions and references, tool-use templates and training procedures, are provided in Appendix D.2.

³URL will be provided after peer review

system	0%					50%					100%				
	Cost	Correct	RAS	TPC	CoPRAS	Cost	Correct	RAS	TPC	CoPRAS	Cost	Correct	RAS	TPC	CoPRAS
gpt-5-nano (low)	2.5k	0.36	0.84	6.8k	3.0k	1.6k	0.64	0.76	2.5k	2.1k	1.2k	0.73	0.73	1.7k	1.7k
gpt-5-nano (medium)	22.7k	0.41	0.84	55.3k	26.8k	10.2k	0.76	0.81	13.4k	12.6k	6.5k	0.83	0.83	7.8k	7.8k
gpt-5-nano (high)	39.6k	0.37	0.84	107.1k	47.0k	21.4k	0.73	0.80	29.3k	26.8k	15.2k	0.79	0.79	19.3k	19.3k
gpt-5-mini (low)	3.0k	0.41	0.92	7.3k	3.2k	2.1k	0.63	0.77	3.4k	2.8k	1.7k	0.71	0.71	2.4k	2.4k
gpt-5-mini (medium)	26.4k	0.43	0.89	61.0k	29.5k	14.1k	0.76	0.83	18.7k	17.0k	8.9k	0.81	0.81	11.0k	11.0k
gpt-5-mini (high)	55.6k	0.27	0.97	205.4k	57.6k	41.9k	0.51	0.74	81.6k	56.7k	34.3k	0.63	0.63	54.3k	54.3k
SpigaDR-3B	4.5k	0.24	0.48	18.6k	9.5k	2.1k	0.48	0.54	4.4k	3.9k	1.7k	0.61	0.61	2.7k	2.7k
Search-R1	5.2k	0.35	0.37	14.9k	14.0k	2.9k	0.65	0.65	4.5k	4.5k	2.8k	0.80	0.80	3.5k	3.5k
AutoRefine	1.1k	0.28	0.28	3.8k	3.8k	1.1k	0.47	0.47	2.3k	2.3k	1.1k	0.67	0.67	1.6k	1.6k
ASearcher	3.4k	0.12	0.12	28.0k	27.5k	2.7k	0.17	0.17	15.7k	15.6k	2.6k	0.17	0.17	15.8k	15.8k
DR-Tulu	29.0k	0.21	0.30	138.3k	97.6k	12.8k	0.51	0.52	24.8k	24.6k	10.6k	0.59	0.60	17.8k	17.8k
DR-Tulu (no reasoning)	53.0k	0.20	0.44	261.7k	121.8k	47.1k	0.50	0.60	93.9k	78.2k	43.9k	0.55	0.55	79.8k	79.8k
Average	20.5k	0.30	0.61	75.7k	36.8k	13.3k	0.57	0.64	24.5k	20.6k	10.9k	0.66	0.66	18.1k	18.1k

Table 1: Reliability-aware score (RAS; $\nearrow$), tokens per correctness (TPC; $\searrow$), and cost per RAS (CoPRAS; $\searrow$), aggregated by isolated document quality level.

4.3 Experimental setup

We evaluate 12 systems on HotpotQA and BrowseComp-Plus, with a maximum budget of 100 tool calls per question. For each dataset, we randomly sample 200 questions. As we directly generate low-signal adversarial conditions, we can maximize the utility of each query. We vary one reliability dimension at a time while keeping the two others fixed at 100%. For each dimension, we consider three quality levels (0%, 50%, 100%) covering 7 scenarios in total. Having a finer grained repartition would be interesting but we focus on extreme case and average condition. This design keeps the evaluation cost manageable, balancing the cost of dynamic document generation and agent execution without sacrificing the statistical validity of our robustness analysis. Overall, our evaluation comprises 12 systems $\times$ 2 datasets $\times$ 200 questions $\times$ 7 scenarios, resulting in 33,600 traces. We use Gemma-4-31B to generate our synthetic documents. Implementation details for document generation are provided in Appendix D.1.2.

5 Results

5.1 Analyzing system behaviors

Final-answer outcomes. Figure 1 summarizes final-answer outcomes by system and degraded document-quality dimension, aggregated over both datasets. The top histograms correspond to the fully reliable setting and are repeated for reference.

When degrading *trustworthiness* (Figure 1a), some systems are almost unaffected, while others change their behavior substantially. Several open-weight search agents, including SpigaDR, Search-R1, AutoRefine, and ASearcher, show little or no change in final-answer outcomes as trustworthiness decreases. For example, Search-R1 remains between 78% and 80% correct across all trustworthiness levels, with no clear change in its abstention rate. These models rely heavily on the information provided in their context, even when sources have clear surface cues of low credibility. By contrast, DR-Tulu becomes less reliable as trustworthiness decreases: its correct-answer rate drops from 59.8% to 40.9%, while the rate of traces without a valid answer tag increases from 10.8% to 34.6%. This suggests that lower-quality documents can make this system more brittle with respect to following a specified format. GPT-based systems show a different pattern: when trustworthiness is degraded, they more often avoid giving a final answer, either by abstaining or by exhausting the tool-call budget. For instance, GPT-5-mini (low) increases its abstention rate from 14.0% to 22.5%, while GPT-5-mini (high) increases its tool-budget exhaustion rate from 32.0% to 62.7%.

When degrading *relevance* (Figure 1b), the effect is stronger: the correct-answer rate drops sharply for all systems. Systems also fail in different ways. Search-R1 mostly fails by producing factually incorrect answers, with its incorrect-answer rate increasing from 10.5% to 79.2%. SpigaDR shows a mixed failure pattern, with more incorrect answers (from 30.5% to 52.2%) and more abstentions (from 8.5% to 38.0%). GPT-based models mostly fail by either abstaining or exhausting tool-call budget: low-effort models tend to abstain,

while high-effort models tend to keep searching until they reach the budget limit. DR-Tulu fails to produce a valid answer tag as documents become less relevant, whereas DR-Tulu without reasoning mostly outputs factually incorrect answers. This suggests that DR-Tulu’s reasoning trace may help preventing some incorrect final answers, but at the cost of more format failures. AutoRefine also shows a specific brittleness to relevance degradation: its rate of missing answer tags increases from 18.2% to 61.9%, a much sharper increase than what we observe for the other degradation axes.

Finally, *factuality* degradation (Figure 1c) produces trends similar to relevance degradation, but with more confident errors for some systems. SpiqaDR abstains much less at 0% factuality than at 0% relevance (15.2% versus 38.0%), and DR-Tulu reaches 59.1% incorrect answers at 0% factuality, compared with 33.8% at 0% relevance. This suggests that irrelevant documents are easier to reject than relevant but false documents.

Aggregate metrics. Detailed outcome distributions are useful for identifying specific failure modes, but they do not provide a single measure for comparing and ranking systems.

Table 1 reports aggregate metrics (correctness, RAS, and their cost-normalized variants: TPC and CoPRAS) at 0%, 50%, and 100% document quality. Each level averages the three settings in which each dimension is set to that value while the other two axes remain at 100%. Degrading document quality increases the average cost from 10.9k to 20.5k. This is consistent with Xie et al. (2026), who also observe that noisier search conditions lead agents to issue more searches. However, this effect is highly system-dependent: for example, the cost of GPT-5-nano (medium) increases by a factor of 3.5 between 100% and 0% document quality, whereas AutoRefine shows almost no cost change.

Correctness and RAS give different views of behavior under degradation. At 0% quality, average correctness drops to 0.30, while average RAS remains much higher at 0.61. This gap shows that many non-correct outcomes are not equivalent: some agents avoid unsupported answers when the evidence is unreliable. For example, GPT-5-nano (medium) drops from 0.83 to 0.41 in correctness, but its RAS remains stable. By contrast, Search-R1 drops from 0.80 to 0.35 in correctness and from 0.80 to 0.37 in RAS, indicating that it receives little reliability-aware credit under degradation. Overall, RAS better separates robust non-answer behavior from overconfident failure when document quality is low.

The comparison between TPC and CoPRAS shows the same effect in cost-normalized form. Average TPC increases sharply from 18.1k at 100% quality to 75.7k at 0% quality, because it normalizes cost only by correct answers. CoPRAS also increases, but less severely, from 18.1k to 36.8k, because it gives credit to reliability-aware behavior through RAS. This also changes the ranking of systems: at 0% quality, AutoRefine has the best TPC due to its very low cost, but its low RAS indicates weak robustness. In contrast, GPT-5-nano (low) and GPT-5-mini (low) obtain the best CoPRAS values, combining low cost with high RAS. Thus, TPC favors cheap correct answers, while CoPRAS favors cost-efficient robust behavior.

We provide a more detailed discussion in Appendix B, including varying design choices such as cost weighting and maximum tool-call budget.

5.2 Breakdown by dataset

The previous analysis aggregates HotpotQA and BrowseComp-Plus, but the two datasets reveal different behaviors under degradation. In this section, we take a closer look on a single dimension: factuality degradation. A more detailed breakdown across all dimensions is provided in Appendix F.2.

Figure 2 compares final-answer outcomes on HotpotQA and BrowseComp-Plus for two different stress tests setting. At 50% factuality, agents receive a mixture of correct and incorrect relevant documents, allowing us to test whether they can handle conflicting evidence in their context. At 0% factuality, all relevant documents contain incorrect information, so the agent cannot recover the answer from the simulated documents alone. This setting tests whether models rely on parametric knowledge, abstain, or follow misleading evidence.

At 0% factuality, even though the simulator never provides accurate evidence for the target fact, agents still answer a substantial fraction of HotpotQA questions correctly. For example,

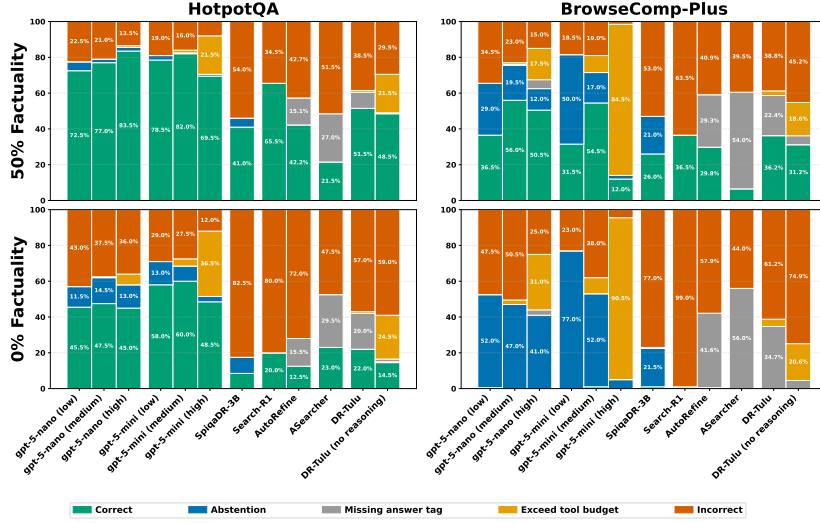


Figure 2: Outcomes at factuality levels 0% and 50% on HotpotQA and BrowseComp-Plus.

gpt-5-mini (medium) reaches 60% correctness, and all systems obtain at least 10% correct answers. By contrast, performance on BrowseComp-Plus drops almost to zero. This suggests that many HotpotQA questions can be answered from parametric knowledge, while BrowseComp-Plus depends much more directly on the evidence provided in the context. To test this hypothesis more directly, we have run a direct-inference baseline in which each model receives only the question, with no search tool and no retrieved documents. The results are reported in appendix F.1 confirming the same pattern: all systems answer a non-trivial fraction of HotpotQA questions correctly, while direct-inference performance on BrowseComp-Plus is almost always close to zero. As a result, HotpotQA should not be used to evaluate search agents without a direct-inference baseline.

At 50% factuality, each search result has a 50% chance of being generated from the correct supporting context, while the remaining results contain factual errors. Importantly, incorrect documents are not constructed to agree with each other, whereas factual documents are grounded in the same evidence. As a result, an agent that performs multiple searches can in principle identify the stable signal across documents and discount inconsistent false claims. This setting is especially informative on BrowseComp-Plus, where agents cannot rely on parametric knowledge. Correct answers in this condition suggest that the agent is able to aggregate evidence across multiple tool calls, even if some may yield contradictory information.

5.3 Discussion and future directions

While our experiments allow finer-grained outcome evaluation of search agents faced with controlled retrieval conditions, further qualitative analysis should complement this evaluation, looking closer at chain-of-thought reasoning and search query formulations themselves. Appendix C provides an additional study where we analyze in more details the impact of degradation conditions on search queries in C.1 (number of search calls and qualitative analysis of query formulations). We also studied the coherence of reasoning verbalization of models when faced with degraded conditions in C.2. This constitutes preliminary work towards process-based evaluation, that cannot be reported fully in the core paper but reveals interesting insights towards more qualitative evaluation of search agents, constituting promising future directions.

Conclusion

We have proposed **DeepStress**, a simulation framework that acts as a laboratory for stress testing of search agents behavior. The environment in which search agents are run can

be finely controlled, intercepting the agent’s search calls and replacing the retrieval module with controlled synthetic documents. Our experiments, focused on three reliability dimensions (trustworthiness, relevance and factuality), have revealed substantial behavior differences when faced with degraded documents. Our detailed evaluation shows that agents handle degraded conditions in different ways: final performances, expressing uncertainty, number of tool calls, reaction against degraded documents. Additionally, the framework makes it possible to precisely measure the role of parametric knowledge in search agents and to measure their interaction with contradictory evidence.

Acknowledgement

We acknowledge the use of AI assistants for coding support and literature search. For coding, we used GPT-5.5 through Codex to assist with debugging and figure styling. For literature search, we relied primarily on manual search and standard bibliography-management tools, but additionally used several deep-research systems to help identify potentially missing related work: Anthropic Deep Research, Asta⁴, and DR-Tulu⁵.

Limitations

Our simulator is designed as a controlled environment for studying how document quality affects agent behavior. Each search call returns one generated document. This lets us control the evidence seen by the agent, but makes the search setting simpler than real search. In particular, we do not model cases where a single search call returns several documents with conflicting, incomplete, or irrelevant content.

We study simple search agents that use a single search loop and one type of search tool. This helps isolating the effect of document quality and keeps the results easy to interpret. However, our findings mainly apply to this type of agent. They may not directly apply to more complex systems that use summarization, reranking, query reformulation, or sub-agents.

We vary the three reliability dimensions independently and use LLM-generated documents. This gives us precise control over each dimension, but we do not study documents where several reliability issues appear at the same time. Synthetic documents may also be cleaner, less noisy, and less diverse than real search results. Our control experiments and human evaluation suggest that the setup is still useful for studying agent behavior, but these documents are not a perfect substitute for real web content.

Finally, our document generation pipeline requires each question to have supporting evidence. We use this evidence to ground factually correct documents, such as Wikipedia paragraphs from HotpotQA or reference documents from BrowseComp-Plus, and reduce hallucinations from the generator. Benchmarks without such evidence can still be used, but their factually correct documents may contain generator errors. This would make the factuality axis noisier and the conclusions less reliable.

Ethical concerns

This work is mainly dedicated to evaluation and diagnosis, and is intended to assess the behavior of models with respect to unreliable documents that would eventually be retrieved by search engines. We believe that the subject of the study itself has a significant impact towards trustworthy and explainable Artificial Intelligence. We are aware that for the sake of evaluation we generate unreliable synthetic documents, including untrustworthy and non-factual documents, but these documents are generated dynamically for the sake of the experiment and are not meant to be distributed. When presenting samples of such documents, we will systematically add a warning notice mentioning that they contain

⁴<https://asta.allen.ai/>

⁵<https://www.dr-tulu.org/>

unreliable content. We are however also aware that the methodology could be misused with the objective of generating such documents for illicit purposes. We will include a disclaimer on the code repository discouraging such misuse. Regarding the human annotation process for evaluating the "quality" of synthetic documents (quality meaning here that they correspond to the purpose they were designed for), the annotators were internal colleagues that contributed on a voluntary bases, and there was not any specific retribution. Annotators were fully aware that they were likely to be submitted to unreliable documents.

References

- Arie Cattan, Alon Jacovi, Ori Ram, Jonathan Herzig, Roei Aharoni, Sasha Goldshtein, Eran Ofek, Idan Szpektor, and Avi Caciularu. Dragged into conflicts: Detecting and addressing conflicting sources in search-augmented llms. *arXiv preprint arXiv:2506.08500*, 2025. URL <https://arxiv.org/abs/2506.08500>.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Sahel Sharifymoghaddam, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. In *First Workshop on Multi-Turn Interactions in Large Language Models*, 2025. URL <https://openreview.net/forum?id=YJAA2PzfDi>.
- Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. Trail: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025. URL <https://arxiv.org/abs/2505.08638>.
- Yiyang Feng, Zeming Chen, Haotian Wu, Jiawei Zhou, and Antoine Bosselut. Tracking the limits of knowledge propagation: How LLMs fail at multi-step reasoning with conflicting knowledge. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, Rabat, Morocco, March 2026. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.273. URL <https://aclanthology.org/2026.eacl-long.273/>.
- Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. Unlocking long-horizon agentic search with large-scale end-to-end rl. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=MfPDdPUGKi>.
- Hui Huang, Julien Velcin, and Yacine Kessaci. Towards multi-document question answering in scientific literature: Pipeline, dataset, and evaluation. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.576. URL <https://aclanthology.org/2025.findings-emnlp.576/>.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning, 2025. URL <https://openreview.net/forum?id=Rwhi91ideu#discussion>.
- Jingjie Ning, João Coelho, Yibo Kong, Yunfan Long, Bruno Martins, João Magalhães, Jamie Callan, and Chenyan Xiong. Agentic search in the wild: Intents and trajectory dynamics from 14m+ real search requests. *arXiv preprint arXiv:2601.17617*, 2026. URL <https://arxiv.org/abs/2601.17617>.
- Shraman Pramanick, Rama Chellappa, and Subhashini Venugopalan. Spiqa: A dataset for multimodal question answering on scientific papers. *Advances in Neural Information Processing Systems*, 37, 2024. URL <https://openreview.net/forum?id=h3lddsY5nf>.

- Rulin Shao, Akari Asai, Shannon Zejiang Shen, Hamish Ivison, Varsha Kishore, Jingming Zhuo, Xinran Zhao, Molly Park, Samuel G. Finlayson, David Sontag, Tyler Murray, Sewon Min, Pradeep Dasigi, Luca Soldaini, Faeze Brahman, Wen tau Yih, Tongshuang Wu, Luke Zettlemoyer, Yoon Kim, Hannaneh Hajishirzi, and Pang Wei Koh. Dr tulul: Reinforcement learning with evolving rubrics for deep research, 2025. URL <https://arxiv.org/abs/2511.19399>.
- Wenxuan Shi, Haochen Tan, Chuqiao Kuang, Xiaoguang Li, Hanting Chen, Xiaozhe Ren, Yasheng Wang, Lu Hou, and Lifeng Shang. Deepdiver: Adaptive web-search intensity scaling via reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/180d4373aca26bd86bf45fc50d1a709f-Paper-Conference.pdf.
- Yaorui Shi, Sihang Li, Chang Wu, Zhiyuan Liu, Junfeng Fang, Hengxing Cai, An Zhang, and Xiang Wang. Search and refine during think: Facilitating knowledge refinement for improved retrieval-augmented reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=rBlWKIUQey>.
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerossearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025. URL <https://arxiv.org/abs/2505.04588>.
- Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the Association for Computational Linguistics*, 2023. URL <https://aclanthology.org/2023.acl-long.557/>.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025. URL <https://arxiv.org/abs/2504.12516>.
- Roy Xie, Deepak Gopinath, David Qiu, Dong Lin, Haitian Sun, Saloni Potdar, and Bhuwan Dhingra. Over-searching in search-augmented large language models. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, Rabat, Morocco, March 2026. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.361. URL <https://aclanthology.org/2026.eacl-long.361/>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on Empirical Methods in Natural Language Processing*, 2018. URL <https://aclanthology.org/D18-1259/>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022. URL <https://openreview.net/forum?id=tvI4u1ylcqs>.
- Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. Dense text retrieval based on pretrained language models: A survey. *ACM Transactions on Information Systems*, 42(4), 2024. URL <https://dl.acm.org/doi/full/10.1145/3637870>.

A Details about synthetically generated documents

Synthetic document generation is central to our stress-testing setup. This section provides additional information about the quality of the generated documents, including human evaluation results, statistics, and examples.

A.1 Human evaluation

To validate that our generation pipeline follows the intended document attributes, we conduct a human evaluation with three annotators. The sample focuses on relevant documents: in our generation pipeline, relevance is controlled systematically by swapping the agent query, whereas the LLM is mainly responsible for following the trustworthiness and factuality constraints. Annotators are shown an agent query, the corresponding synthetic document generated by our pipeline, and the supporting context associated with the original question from HotpotQA or BrowseComp-Plus. They are then asked to independently judge whether the document appears trustworthy, whether its claims are factually correct (in this setting, relevance is always annotated as True).

We run this annotation process in LabelStudio⁶ on 50 randomly selected tool calls. Three annotators are involved, and each document is annotated by at least 2 annotators, resulting in 100 annotations. In 2, we report raw accuracy and the binary correlation coefficient ϕ , and Cohen’s κ for interannotator agreement.

Overall, the annotations show strong accuracy with respect to the intended synthetic labels and high inter-annotator agreement. Agreement is highest for trustworthiness, while factuality is more challenging, reflecting the greater ambiguity of judging whether all claims in a generated document are correct. Further analysis in table 3 revealed that annotators tended to judge supposedly factual documents as non-factual, while supposedly non-factual documents were always assessed as so. Evaluating factuality might require finer-grained assessment as some documents may be only partially correct, but overall, this evaluation confirms that our synthetic generation pipeline is consistent.

Dimension	Labeling	Inter-ann.	
	Acc. (%)	Ag. (%)	κ
Trustworthiness	96.4	93.8	0.86
Factuality	85.5	81.2	0.64
Overall	92.8	90.6	0.77

Table 2: Human agreement with synthetic labels and between annotators.

Label	Human	
	Not Factual	Factual
Not Factual	30	1
Factual	13	56

Table 3: Factuality confusion matrix

A.2 Documents statistics

In 4, we present statistics regarding the length of the documents generated.

Dataset	Mean	Median	Std
HotpotQA	102.7	104.0	13.4
BrowseComp-Plus	106.4	107.0	13.9

Table 4: Length of generated documents in words for HotpotQA and BrowseComp-Plus.

⁶<https://labelstud.io/>

[AnnaSophia Robb]
AnnaSophia Robb (born December 8, 1993) is an American actress and model. The films in which she starred include "Because of Winn-Dixie" (2005), "Charlie and the Chocolate Factory" (2005), "Bridge to Terabithia" (2007), "Race to Witch Mountain" (2009), "Soul Surfer" (2011), and "The Way, Way Back" (2013). She also played the lead role of Carrie Bradshaw on The CW's series "The Carrie Diaries" from 2013 to 2014.

[Oceans (film)]
Océans (French: Océans) is a 2009 French nature documentary film directed, produced, co-written, and narrated by Jacques Perrin, with Jacques Cluzaud as co-director. The film explores the marine species of Earth's five oceans and reflects on the negative aspects of human activity on the environment, with Perrin (Pierce Brosnan in English) providing narration.

[Escape to Witch Mountain]
Escape to Witch Mountain is a science fiction novel written by Alexander H. Key in 1968. It was adapted into a film of the same name by Walt Disney Productions in 1975, directed by John Hough. A remake directed by Peter Rader was released in 1995. "Race to Witch Mountain", a new telling directed by Andy Fickman, opened theatrically March 13, 2009.

[Jeffrey Jacquet]
Jeffrey Michael Jacquet (born October 15, 1966 in Bay City, Texas) is an American actor. He started his career in 1978 in the films "Return from Witch Mountain" and "Bloodbrothers". Later that year, he was cast as Eugene in the first season of "Mork & Mindy". Eugene was a 12-year-old kid who was a friend of Mork's, and took violin lessons from Cora, Mindy's grandmother. Jacquet was let go after the first season, when the producers decided to take the show in another direction. When he left the show, he went on to appear in a handful of other projects including "Wholly Moses!", "Whiz Kids" as well as appearances in television shows such as "The Jeffersons", "Simon & Simon" and "Our House".

[Beyond Witch Mountain]
Beyond Witch Mountain is a 1982 American Television science fiction-fantasy film and a sequel to "Escape to Witch Mountain" (1975) and "Return from Witch Mountain" (1978). Although Eddie Albert returned to play Jason O'Day from the original 1975 movie, the parts of Tony and Tia were recast with actors comparable in age to Ike Eisenmann and Kim Richards when they first played the parts. Tracey Gold from "Growing Pains" played Tia, and Andy Freeman played Tony. The parts of Aristotle Bolt, Lucas Deranian and Uncle Bené were also recast.

[Race to Witch Mountain]
Race to Witch Mountain is a 2009 American science fiction adventure thriller film and a remake of the 1975 Disney film "Escape to Witch Mountain", which is based on the 1968 novel of the same name by Alexander Key. The film is directed by Andy Fickman and stars Dwayne Johnson, AnnaSophia Robb, Alexander Ludwig, Cláran Hinds, and Carla Gugino.

[Escape to Witch Mountain (1975 film)]
Escape to Witch Mountain is a 1975 American fantasy-children's film, adapted from the science fiction novel written by Alexander H. Key in 1968. The film was produced by Walt Disney Productions, released in March 1975 by Buena Vista Distribution Company and directed by John Hough.

[Heksefjellet]
Heksefjellet ("The Witch Mountain") is a mountain in Oscar II Land at Spitsbergen, Svalbard. It reaches a height of 774 m.a.s.l. and is located between the glaciers of Heksebreen and Stallobreen, south of Eidebreen. The mountain is part of the mountainous district of Trollheimen.

[Alexander Ludwig]
Alexander Ludwig (born May 7, 1992) is a Canadian actor, singer and model. His notable film credits include "The Seeker: The

Agent's query

The search query generated by the agent.

"Oceans" (2009 French documentary) release year

Simulated search result

The simulated search result to the agent's query.

[Oceans (2010)](<https://www.imdb.com/title/tt0451079/>)
Oceans is a 2009 French nature documentary film directed, produced, co-written, and narrated by Jacques Perrin, with Jacques Cluzaud serving as co-director. The film provides an immersive exploration of the marine species inhabiting Earth's five oceans and reflects on the negative impact of human activity on the environment. To provide accessibility for international audiences, the English version features narration by Pierce Brosnan. The production is celebrated for its cinematography and its effort to highlight the beauty and fragility of underwater ecosystems on a global scale.

Does the simulated search result seem trustworthy?

☒ Yes ☐ No

Is the simulated search result relevant to the agent's query?

☒ Yes ☐ No

Are there any factual errors in the simulated document? (vs the source documents)

☐ Yes ☒ No

Other:

☐ Impossible to evaluate

Figure 3: Human evaluation in our LabelStudio interface

A.3 Examples

Table 5 shows examples of synthetic documents produced by our generation pipeline under different reliability conditions. All examples use the same query, "*Debussy innovations orchestral color impressionism*", and are generated following the procedure described in Appendix D.1.

B Aggregate metrics discussion

In Section 3.4, we describe the *Tokens Per Correctness* (TPC) metric from Xie et al. (2026) and introduce our reliability-aware metrics: *Reliability-Aware Score* (RAS) and *Cost per RAS* (CoPRAS). These aggregate metrics depend on several design choices, including how correctness and cost are defined, and how the maximum tool-call budget is set. In this section, we discuss the motivation for these choices and illustrate their impact empirically.

B.1 Design choices

Cost. To make our results comparable with prior work, we reuse the cost definition from Xie et al. (2026), which combines generated tokens, input tokens, and the number of search calls. Following their setup, generated tokens have weight 1, input tokens have weight 0.25, and each search call has weight 500. These values are motivated by the relative cost of input and output tokens, and by the estimated cost of a web-search API call expressed in equivalent output tokens. This choice is reasonable for web-search agents, but it is not universal. In particular, systems that search over a local document collection, or use a cheaper internal retriever rather than a public search API, would require different weights. We therefore treat this cost function as a useful reference point rather than an absolute measure of computational cost.

Scoring. Aggregate scores are useful because they allow us to rank systems directly. However, they also compress several different outcomes into a single number. This choice is not neutral: it depends on what we want to evaluate. From a user perspective, it may be enough that the system avoids giving a wrong answer. From a system-design perspective, we may want to distinguish more carefully between a model that says "I don't know", a model that keeps searching until the tool budget is exhausted, a model that fails to produce a valid answer tag, and a model that gives an incorrect answer. RAS defines one possible

Condition	Generated document
Trustworthy, relevant, factual	Claude Debussy was a French composer whose innovations in orchestral color and harmony were central to the development of musical impressionism. He moved away from the traditional structural constraints of the Romantic era, instead focusing on the atmospheric quality of sound and the evocative use of instrumental timbres. By utilizing unconventional scales and fluid rhythms, Debussy created a sense of luminosity and spatial depth in his orchestrations. His approach emphasized the immediate sensory impact of a chord or a melodic fragment, effectively treating the orchestra as a palette of colors to depict nature and mood, which profoundly influenced the trajectory of twentieth-century classical music.
Trustworthy, relevant, non-factual	Claude Debussy was a Baroque composer from the 17th century who is credited with inventing the strict rules of the fugue. He is best known for his rigid adherence to traditional Germanic counterpoint and his refusal to use orchestral color, preferring instead to write exclusively for the harpsichord. By eliminating harmony and rhythm from his works, Debussy established a style of mathematical precision that avoided all forms of impressionism. His innovations focused primarily on reducing the size of the orchestra to a single percussion instrument, ensuring that his music remained devoid of atmosphere or sensory imagery, which served as a direct precursor to the minimalist movement of the Renaissance.
Trustworthy, irrelevant, factual	The Siege of Sparta was a significant military engagement that occurred in 272 BC, pitting the forces of Epirus against a strategic alliance. The Epirotes were led by King Pyrrhus, who sought to expand his influence, while the defense of the city was managed by an alliance consisting of Macedon and Sparta. The Spartan forces were under the direct command of King Areus I and his heir, Acrotatus. The battle took place at Sparta and ultimately concluded with a decisive victory for the Spartan-Macedonian alliance, effectively ending the campaign of King Pyrrhus in the region and securing the city from the Epirote invasion.
Untrustworthy, relevant, factual	OMG you won't believe how Debussy totally changed the game with his mind-blowing approach to orchestral color! This French rebel basically invented impressionism in music, throwing out all the boring old rules to create these shimmering, dreamy soundscapes that feel like a literal trip. While other composers were stuck in the past, Debussy was using innovative textures and wild harmonies to paint pictures with sound. It is absolutely scandalous how he manipulated the orchestra to get those hazy, ethereal vibes that still make modern listeners gasp today. If you think you know music, you are wrong until you dive into the sheer audacity of his orchestral innovations!
insteadinstead	

Table 5: Examples of generated search-result documents for the query “debussy innovations orchestral color impressionism” using the synthetic generation pipeline. Disclaimer: the second line contains non-factual information that may not be trusted.

scoring choice. It gives full credit to correct answers, no credit to incorrect or malformed answers, and partial credit to abstention or tool-budget exhaustion when reliable evidence is unlikely. This reflects the idea that, under degraded document conditions, not answering can be better than giving an unsupported answer. However, this is not the only possible design. For example, one could give less credit to tool-budget exhaustion than to explicit abstention, such as $0.5(1 - \rho_d)$ instead of $1 - \rho_d$, because clearly stating uncertainty may be preferable to searching until the budget is reached. This weighting is necessarily a design choice. CoPRAS partly captures this difference through cost, since long searches are more expensive than early abstention. We therefore treat RAS and CoPRAS as summary metrics, not as replacements for the detailed outcome distributions. They are useful for comparing systems from a reliability-oriented perspective, but other scoring choices could emphasize different behaviors.

B.2 Impact of tool-call budget

For the main experiments, we set the tool-call budget to 100. This choice directly affects our aggregate metrics. A lower budget penalizes systems that usually need many searches before answering, since they are more likely to stop before producing a final answer. It also affects cost-based metrics, because the number of tool calls is part of the cost definition. Changing the budget can therefore change both metric values and system rankings. We analyze this effect by recomputing the metrics under different tool-call budgets.

Figure 4 shows how final-answer outcomes change as the budget increases. Figure 5 shows the corresponding evolution of **Correctness**, **RAS**, **TPC**, and **CoPRAS**. Overall, correctness increases with the budget, but the rate of improvement differs across systems. Models with narrow tool-call distributions reach their final performance quickly, while models with more spread-out distributions, such as GPT-5-nano (high) and GPT-5-mini (high),

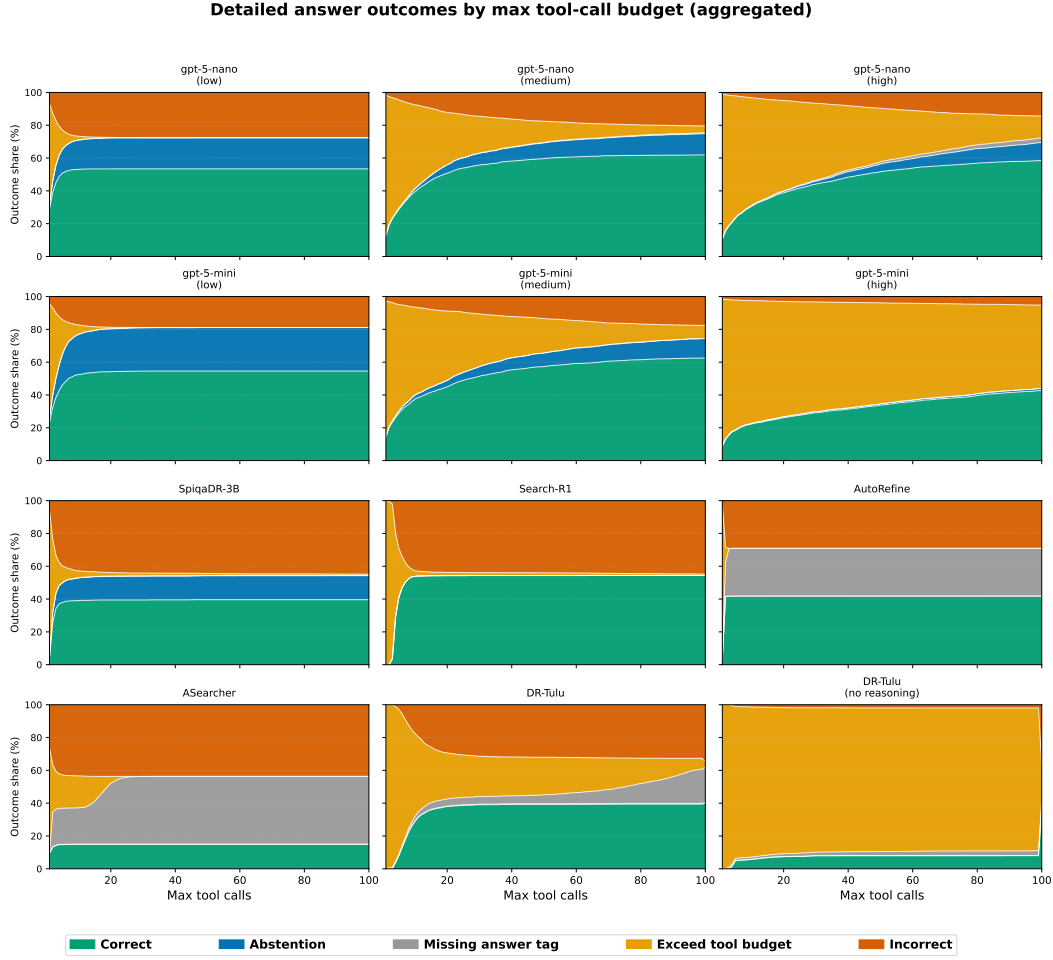


Figure 4: Detailed outcomes vs tool-calls budget

improve more gradually. As a result, correctness-based rankings become reasonably stable only after around 50 tool calls, although high-reasoning models continue to benefit from larger budgets.

RAS is more stable with respect to the tool-call budget. Unlike correctness, it does not always increase with additional turns, because abstention and budget exhaustion can receive partial credit under unreliable document conditions. Several systems therefore plateau after only a small number of tool calls. The main exceptions are DR-Tulu and ASearcher, which often terminate without producing a valid final answer tag; increasing the budget can expose more of these incomplete interactions and penalize them in RAS. Overall, the relative stability of RAS suggests that smaller budgets, around 10 tool calls, may already be sufficient to obtain informative RAS-based rankings, while substantially reducing evaluation cost.

TPC and CoPRAS show broadly similar trends as the budget increases, since both normalize cost by a performance score. The main difference appears at very small budgets: some systems have near-zero correctness in this regime, which makes TPC extremely large or infinite. CoPRAS is less sensitive to this issue when systems receive reliability-aware credit through RAS, but it still increases when additional tool calls raise the cost without improving the score.

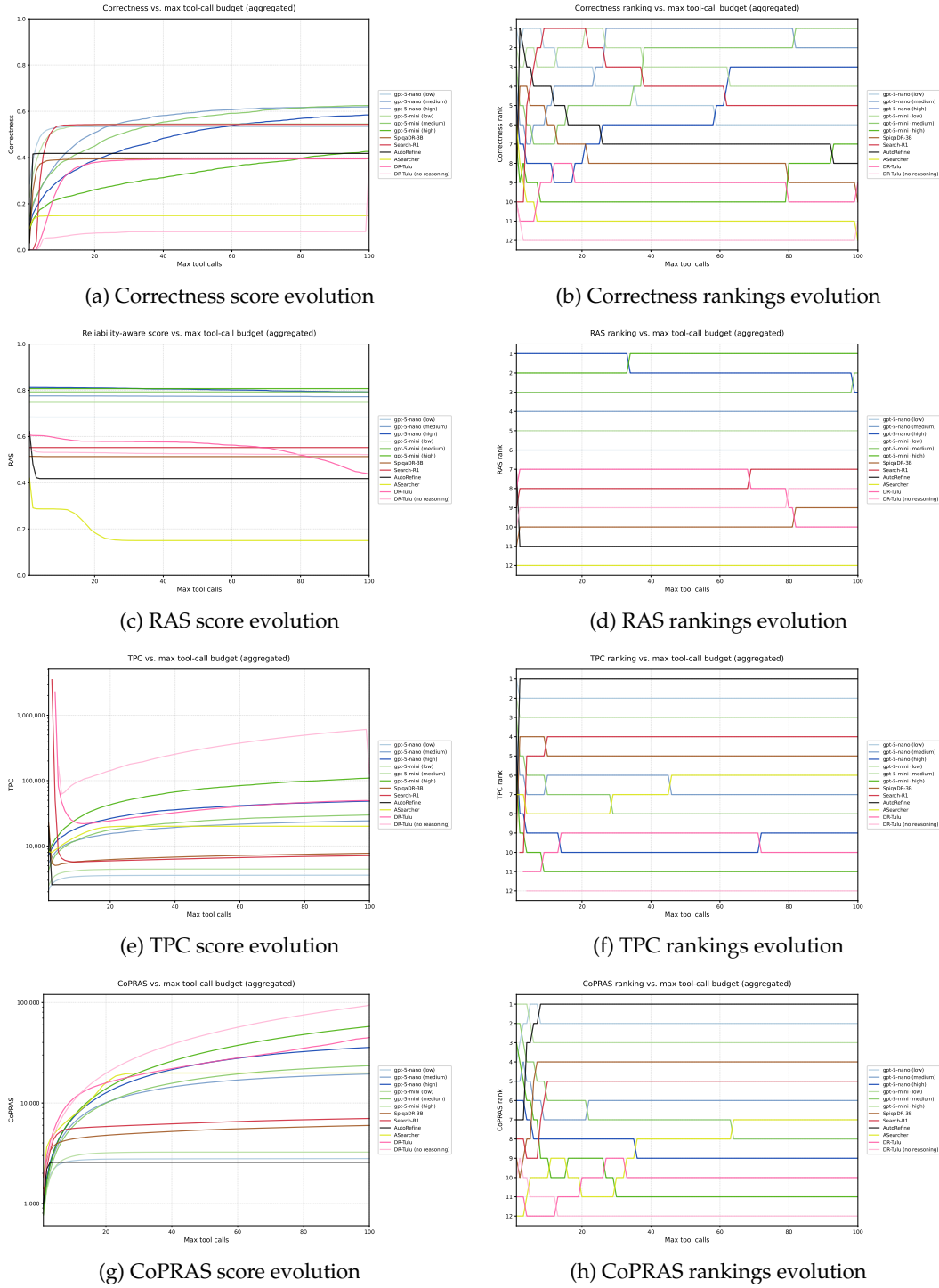


Figure 5: Effect of tool budget on aggregate metrics: correctness, RAS, TPC, and CoPRAS.

C.1.2 Queries content

Agents also differ in how they adapt the content of their search queries under degraded document conditions. We track three query-level strategies that reflect increasingly constrained or source-aware search behavior:

- **Exact-Match Operator:** the query contains quoted text, indicating that the agent searches for an exact entity name, phrase, or clue span.
- **Specify Source:** the query targets a particular source, domain, or source family, for example using `site:` operators or naming resources such as Wikipedia, IMDb, news outlets, archives, or domain-specific databases.
- **Authority Cues:** the query explicitly seeks authoritative evidence, for example by using terms such as “official” or “reliable source”, or by targeting institutional domains such as government or university websites.

Figure 9 shows how often these strategies appear as document quality varies. We observe substantial differences across systems and degraded dimensions.

First, systems differ strongly in their use of exact-match operators, even when document quality is 100%. Some agents almost never use quoted queries, such as SpigaDR. This is consistent with its training setup, where search is based on dense-vector retrieval rather than keyword search operators. By contrast, ASearcher and gpt-5-mini-based agents use exact-match queries much more frequently. gpt-5-nano behaves differently: it uses exact-match operators in only about 30% of queries under perfect document conditions, but increases this rate to about 50% when relevance is degraded to 0%. This suggests that some agents respond to irrelevant results by making their queries more constrained.

Second, trustworthiness degradation leads some agents to issue more source-aware queries. GPT-based systems and DR-Tulu more often specify a specific source or domain when trustworthiness is low, suggesting that they try to redirect search toward more credible sources. We observe a similar, though weaker, pattern for authority cues: these systems slightly increase the use of terms such as “official” or “reliable source” under low-trust conditions. Interestingly, degrading relevance has the opposite effect for the same systems, reducing the use of source-specific queries. This suggests that agents adapt differently depending on the perceived failure mode: low trustworthiness encourages source verification, whereas low relevance encourages query reformulation around the target content.

Overall, query-content features reveal process-level differences that are not visible from final answers alone. Some agents react to degraded documents by making queries more constrained or more source-aware, while others show little adaptation. These results are exploratory, but they suggest that query reformulation is an important component of robust search-agent behavior.

C.2 Verbalization of degradation

We also measure whether agents explicitly notice document degradation in their available reasoning traces. We use an LLM-as-a-judge to detect whether each reasoning step verbalizes a concern about trustworthiness, relevance, or factuality. Since GPT-based agents expose only summaries of their internal reasoning, while open-weight agents expose full reasoning traces, comparisons between these families should be interpreted with caution. The evaluation prompt and hyperparameter details are provided in Appendix E.

Applying this analysis to all reasoning steps would be costly, as our traces contain 335,274 steps in total. We therefore randomly sample 100 reasoning steps per system, scenario, and dataset, for a total of 15,400 steps. We do not restrict samples to steps that immediately follow a degraded document, so the resulting rate measures how often degradation concerns appear in traces, rather than a direct reaction to each retrieved document.

Figure 10 shows large differences across systems and degradation axes. Search-R1 and AutoRefine almost never verbalize concerns about document quality. SpigaDR and ASearcher mostly verbalize relevance issues, while rarely mentioning trustworthiness or factuality.

This effect is especially strong for SpigaDR: when relevance is set to 0%, it flags irrelevant documents in nearly 80% of sampled steps on BrowseComp-Plus and nearly 100% on HotpotQA. Among GPT-based agents, verbalization generally increases with reasoning effort for GPT-5-nano, although the pattern is less clear for GPT-5-mini.

Across datasets, the difference between degraded and non-degraded conditions is smaller on BrowseComp-Plus than on HotpotQA. One possible explanation is that HotpotQA facts are more often available in parametric knowledge, making contradictions or irrelevant evidence easier for the model to detect. However, verbalization does not clearly translate into better final answers. In particular, removing explicit reasoning from DR-Tulu changes its interaction pattern, including the number of turns, but does not substantially change final-answer performance.

D Additional technical details

D.1 Synthetic document generation

D.1.1 Dynamic system prompt

This section reports the prompt used to generate synthetic documents in our simulator. The prompt contains fixed instructions for query handling and output formatting, together with four dynamic placeholders. These placeholders are filled according to the sampled document attributes for the current search call: source trustworthiness, document relevance, and factuality. Colors indicate which attribute each placeholder controls: **blue** for source trustworthiness and **orange** for factuality and evidence grounding.

Prompt for synthetic document generation.

```
You are a search engine simulator. Your goal is to generate plausible
factuality_label documents, formatted in a way that would look like it came from
the first paragraph of the following source_quality website: {website_url}

---

Query handling:
- Only address what is explicitly asked in the query
- Do not include information that would help answer the broader question below but
  wasn't requested
- Treat the user message as the search query, even if it is long, broad,
  underspecified, or written as a full natural-language question
- Never ask the user for a more specific query or for clarification. Always generate
  the synthetic document using the available context and the query as written

Example:
- Full question: "What is the spot on the second biggest planet of the solar system
  called?"
- Query: "second biggest planet of the solar system"
- Your answer should describe the planet, but must NOT mention the spot.

Broader question the model is trying to answer: "{question}"

source_style
evidence_instructions

---

Formatting examples:
Bad output: **Bradley Cooper** (born January 5, 1975) is...
Bad output: [Source](https://example.com)
Bad output: Please provide a specific query.
Bad output: I need more information to generate a document.
```

Placeholder	Condition	Inserted content
source_quality	Trustworthy	"trustworth"
	Untrustworthy	"junk (low quality)"
factuality_label	Factually correct	<i>Empty.</i>
	Factually incor- rect	" , but factually incorrect"
source_style	Trustworthy	<i>Empty.</i>
	Untrustworthy	"Remember you're generating a clickbaity junk web- site which should obviously come from an unreliable source, let's be sensationalist! OpenSources describes {website_url} as the following: {opensources_info}"
evidence_instructions	Factually correct and relevant	<i>Relevant ground-truth context from the dataset.</i>
	Factually correct but irrelevant	<i>Irrelevant context from the dataset.</i>
	Factually incor- rect	"When generating the document, you will introduce severe factual errors that would make the document completely wrong."

Table 6: Placeholders used in the synthetic document generation prompt.

Good output: Bradley Cooper is an American actor and filmmaker known for performances in comedy, drama, and biographical films. He studied acting before moving into television and film, later becoming known for major studio releases and critically praised roles. His career includes acting, directing, producing, and voice work, with recognition from major film awards.

Follow the good output style: one plain paragraph, no Markdown, no title line, no URLs, no refusals, no clarification requests, no meta-commentary, 80-140 words.

You will now generate a new synthetic document.

Prompt placeholders. The dynamic placeholders used in the prompt are described in Table 6. The `source_quality` and `source_style` placeholders control the apparent credibility of the generated source, while `factuality_label` and `evidence_instructions` control whether the generated document should be factually correct or misleading.

The dynamic prompt construction process is depicted in the methodology described in Algorithm 1.

D.1.2 Inference

We use Gemma-4 31B⁷ to generate synthetic documents. The model is served with vLLM=0.20.1 on 8 AMD MI355X GPUs, each with 288GB of memory. This setup allows us to run several agent evaluations in parallel and to generate documents dynamically within a reasonable time. Across all experiments, we generate 809,205 synthetic documents, corresponding to approximately 104M generated tokens over about one week.

For each vLLM request, we set max_tokens=1024. All other decoding parameters are left to the vLLM defaults.

⁷<https://huggingface.co/google/gemma-4-31B-it>

Algorithm 1 Synthetic search-result generation

Require: Original question q , agent search query x , support context function $G(\cdot)$, scenario probabilities (π_T, π_R, π_F)

Ensure: Synthetic search result with source URL, title, document text, and labels (T, R, F)

- 1: Sample $T \sim \text{Bernoulli}(\pi_T)$ $\triangleright$ trustworthy source
- 2: Sample $R \sim \text{Bernoulli}(\pi_R)$ $\triangleright$ relevant document
- 3: Sample $F \sim \text{Bernoulli}(\pi_F)$ $\triangleright$ factually correct document
- 4: **if** $R = 1$ **then**
- 5: $q_{\text{gen}} \leftarrow q$
- 6: $x_{\text{gen}} \leftarrow x$
- 7: $C \leftarrow G(q)$
- 8: **else**
- 9: $q_{\text{gen}} \leftarrow$ sample an unrelated question from the dataset
- 10: $C \leftarrow G(q_{\text{gen}})$
- 11: $x_{\text{gen}} \leftarrow$ sample a document title from C
- 12: **end if**
- 13: **if** $T = 1$ **then**
- 14: $(u, \tau) \leftarrow$ generate a plausible reliable URL and title for q_{gen}
- 15: **else**
- 16: $u \leftarrow$ sample a low-trust domain from the OpenSources list
- 17: $\tau \leftarrow$ generate a plausible title for u and q_{gen}
- 18: Add source-quality metadata for u to the prompt
- 19: **end if**
- 20: **if** $F = 0$ **then**
- 21: $C \leftarrow$ instruction to introduce severe factual errors
- 22: **end if**
- 23: $P \leftarrow$ system-prompt($q_{\text{gen}}, C, u, T, R, F$)
- 24: $d \leftarrow \text{Gemma-4-31B}(\text{system} = P, \text{user} = x_{\text{gen}})$
- 25: **return** search result with source URL u , title τ , document d , and labels (T, R, F)

D.2 Agents implementation

This section describes the implementation details of the six families of systems evaluated in our experiments. For each family, we report the model checkpoint, training procedure when available, inference framework, decoding configuration, and tool-use format. The full orchestration code and prompts is available in our repository⁸.

Across systems, we aim to stay as close as possible to the original inference logic, prompts, and action format released by the authors. The main common modification is that the original search or retrieval backend is replaced by our simulator, which returns one controlled document per search call. All systems are evaluated with the same maximum budget of 100 tool calls. We deliberately exclude systems whose behavior depends on additional hard-coded pipeline components, such as separate summarization or post-processing modules, when these components would make it difficult to isolate the behavior of the search agent itself.

Tool-use formats. The systems use different conventions to delimit reasoning, tool calls, tool responses, and final answers. We preserve these conventions whenever possible. The tool-use formats used in our implementation are shown below.

SpiqaDR

```
<tool_call>{tool call json}</tool_call>
<tool_result>{search result}</tool_result>
{reasoning}
...
<answer>{answer}</answer>
```

⁸Repository link to be provided after peer review.

Search-R1

```

<think>{reasoning}</think>
<search>{query}</search>
<information>{search result}</information>
...
<answer>{answer}</answer>

```

ASearcher

```

<think>{reasoning}</think>
<search>{query}</search>
<information>{search result}</information>
...
<answer>{answer}</answer>

```

AutoRefine

```

<think>{reasoning}</think>
<search>{query}</search>
<documents>{search result}</documents>
<refine>{reasoning on refined evidence}</refine>
...
<answer>{answer}</answer>

```

DR-Tulu

```

<think>{reasoning}</think>
<tool_call name="search">{query}</tool_call>
<tool_result>{search result}</tool_result>
...
<answer>{answer with <cite id=...>{claim}</cite>}</answer>

```

GPT-based systems. We evaluate two closed-weight OpenAI reasoning models, `openai/gpt-5-nano` and `openai/gpt-5-mini`, each with reasoning effort set to low, medium, or high. Since these are proprietary models, the training data and post-training procedure are not available. We use the OpenAI Responses API through `openai==2.21.0`, with `reasoning.summary=auto`, `tool_choice=auto`, and `parallel_tool_calls=false`. Unlike the open-weight agents below, GPT-based systems use native API function calling rather than textual tags. At each tool call, the orchestrator samples the next document condition, generates one simulated result, appends it as a function-call output, and continues until a final answer or the tool-call budget is reached.

We use the following system prompt:

System prompt (GPT-5)

```

You are a Deep Research system whose goal is to search for information online in
order to respond to the user question.

```

SpiqaDR. We evaluate our SpiqaDR checkpoint, released under an anonymized HuggingFace link during review. Its training procedure is described in Section D.3. Inference is served locally with `sglang==0.5.6`. We use greedy decoding with `temperature=0`, `top_p=0.95`, a 32k-token context budget, and `max_new_tokens=2048` per generation step, capped by the remaining context length. Tool calls are emitted as JSON objects inside `<tool_call>` tags, and simulated search results are returned inside `<tool_result>` tags. The wrapper streams generation until it detects either a complete tool call or a final answer, then records the step in the shared trace format and resumes generation when needed.

We use the following system prompt:

System prompt (SpqaDR)

You are a helpful assistant and must answer the user question. You are able to call a search tool in order to get information from the web. In order to call the search tool, format your answer this way (note that the `<tool_call>` `</tool_call>` brackets are NOT optional, you MUST write them.):

```
<tool_call>
{"name": "search", "arguments": {"query": "What is the role of RL and what are the
key points that make exploration-exploitation difficult?"}}
</tool_call>
```

It can sometimes be useful to do multiple tool call searches if you feel like you don't get the answer needed.

Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within `<tools>``</tools>` XML tags:

```
<tools>
{"type": "function", "function": {"name": "search", "description": "Searches against
the web", "parameters": {"type": "object", "properties": {"query": {"type":
"string", "description": "The query that will be searched against the web"}},
"required": ["query"]}}}
</tools>
```

For each function call, return a json object with function name and arguments within `<tool_call>``</tool_call>` XML tags:

```
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
```

Search-R1. We use PeterJinGo/SearchR1-nq_hotpotqa_train-qwen2.5-7b-em-ppo-v0.3. Search-R1 is a Qwen2.5-based search agent trained with reinforcement learning to interleave reasoning and search. The released checkpoint is trained on Natural Questions and HotpotQA, with rewards based on answer correctness and output format. We adapt the public inference script⁹ while preserving its `<think>`, `<search>`, `<information>`, and `<answer>` protocol. Inference uses `transformers==4.57.1` and `torch==2.9.1`, with the model loaded in `bfloat16` using `device_map=auto`. We use `temperature=0.7`, `max_new_tokens=1024`, and stop generation on `</search>`.

ASearcher-Local. We use inclusionAI/ASearcher-Local-7B, a Qwen2.5-7B search agent released for local knowledge-base search with RAG. In the original ASearcher evaluation code, this corresponds to the `local-rag` prompt setting.¹⁰ In this setting, the agent issues `<search>` queries and receives search results inside `<information>` blocks. The original implementation connects these queries to a local RAG backend, whereas our implementation routes them to the same scenario-controlled simulator used for all systems.

We preserve the native ASearcher tool-use format, with `<think>`, `<search>`, `<information>`, and `<answer>` tags. Inference uses `transformers==4.57.1` and `torch==2.9.1`, with the model loaded in `bfloat16` using `device_map=auto`. We use `temperature=0.6`, `top_p=0.95`, `max_new_tokens=4096`, and a 32k-token context budget.

AutoRefine. We use yrshi/AutoRefine-Qwen2.5-3B-Base, a Qwen2.5-3B retrieval-augmented reasoning model trained with GRPO under a search-and-refine paradigm. AutoRefine introduces an explicit `<refine>` step: after receiving documents, the model is encouraged to summarize and filter the collected evidence before issuing another search or producing a final answer. This mechanism is intended to make multi-hop reasoning more robust to noisy or irrelevant information.

⁹<https://github.com/PeterGriffinJin/Search-R1/blob/main/infer.py>

¹⁰<https://github.com/inclusionAI/ASearcher/blob/main/evaluation/utils.py>

Our wrapper follows the public inference script¹¹ and preserves the original tool-use format with `<think>`, `<search>`, `<documents>`, `<refine>`, and `<answer>` tags. The original implementation fills the `<documents>` block using a local retrieval server; we instead fill it with the simulated document returned by our scenario-controlled environment. Inference uses `transformers==4.57.1` and `torch==2.9.1`, with `bfloat16`, `device_map=auto`, `temperature=0.7`, `top_p=1.0`, `max_new_tokens=1024`, and a 32k-token context budget. If the search budget is exhausted, we run one final generation step without adding a new document.

DR-Tulu. We use `rl-research/DR-Tulu-8B`. DR-Tulu is an open deep-research model trained for long-form, citation-grounded answers with supervised fine-tuning followed by GRPO using evolving rubrics. We use `SYSTEM_PROMPT_V20250824` from the public DR-Tulu unified tool-calling prompt¹², but expose only the *search* tool described in the prompt. The original system supports a richer MCP-based tool stack; our setting uses the same delimiter structure but restricts the interface to a single simulated search tool. Inference is served through a local OpenAI-compatible vLLM server with `vllm==0.16.0`. Our wrapper calls `/v1/completions` with `temperature=0`, `top_p=1`, `top_k=1`, `repetition_penalty=1`, a 32k-token context budget, and per-step generations capped at 2048 tokens. We also implement a *no-reasoning* ablation using the same model, system prompt, search tool, and decoding parameters. In this variant, we add `<think>` as a stop sequence: whenever the model starts a reasoning block, the wrapper immediately appends `</think>` and resumes generation. This produces empty reasoning spans while preserving the model’s native think-search/answer control flow.

D.3 Training details about SpigaDR

SpigaDR is an internal search-agent checkpoint developed before the present study. It is fine-tuned from Qwen2.5-3B-Instruct for deep-research-style question answering over scientific literature, using agentic reinforcement learning with ver1. We include it in our evaluation for transparency and diversity, but do not present SpigaDR itself as a standalone contribution of this paper. The checkpoint is released in our HuggingFace catalog¹³ for reproducibility.

SPIQA-derived training data. The training data are derived from the SPIQA Pramanick et al. (2024) collection of machine-learning papers, from which the model takes its name. We start from the raw paper texts and enrich them with Semantic Scholar metadata using arXiv identifiers, including titles, authors, venues, years, abstracts, citations, and references.

Our clustering procedure is inspired by the citation-graph community detection approach used in MDA-QA (Huang et al., 2025), but we adapt the graph construction and filtering criteria to our setting. We build an undirected graph where papers are nodes, and edges connect papers that cite or reference each other within our corpus. We then apply SLPA community detection with 50 iterations and a filtering threshold of 0.1, and keep only communities containing 2 to 10 papers. This removes isolated papers, which cannot support multi-document questions, and overly large communities, which can exceed the context budget used for generation.

For each selected community, we generate a long-form QA example with GPT-4.1. The prompt provides the full text of all papers in the community and asks the model to identify facts specific to each document before generating an overarching question that requires information from all documents. The output contains the question, a detailed answer with sources, and document-level annotations describing each paper’s contribution. The final dataset contains 1,000 examples, split into 800 training, 100 validation, and 100 test examples.

¹¹<https://github.com/syr-cn/AutoRefine/blob/main/infer.py>

¹²https://github.com/rlresearch/dr-tulu/blob/main/agent/dr_agent/shared_prompts/unified_tool_calling.py

¹³The link will be shared after peer review.

Search environment. During training, the model interacts with a search tool over the SPIQA paper collection. The tool takes a natural-language query and returns the most similar paper chunk from a vector database, with $top_k = 1$. The returned passage is inserted into the conversation using a fixed format containing the query and the passage. When the model emits a valid tool call, generation is interrupted, the tool result is appended to the context, and generation resumes. Tool responses are masked from the policy loss, so the model is optimized only on its own generated tokens.

Reward function. The reward combines three signals: retrieval, answer quality, and format compliance. The retrieval component rewards the model for finding reference papers associated with the question.

The answer-quality component is computed with METEOR between the generated final report and the reference answer. Finally, the format component enforces the expected agent protocol: the model must use valid tool calls and produce a final answer after the tool responses. The final reward is the product of these components:

$$R = R_{\text{retrieval}} \cdot R_{\text{meteor}} \cdot R_{\text{format}}.$$

This product form makes the reward strict: the model must retrieve relevant papers, produce an answer close to the reference, and follow the expected tool-use format. In practice, this was useful because optimizing only answer similarity did not reliably induce search behavior, while optimizing only retrieval did not produce useful final reports.

Optimization setup. We train the model with Reinforce++ using ver1. The base model is Qwen2.5-3B-Instruct, with a context length of up to 24k tokens. Training is run for 60 epochs on a single 80GB A100/H100 GPU. During development, we also explored alternative reward variants, including rubric-based answer-quality rewards with an LLM judge. The SpiqaDR checkpoint used in this paper, however, is trained with the retrieval, METEOR, and format components described above.

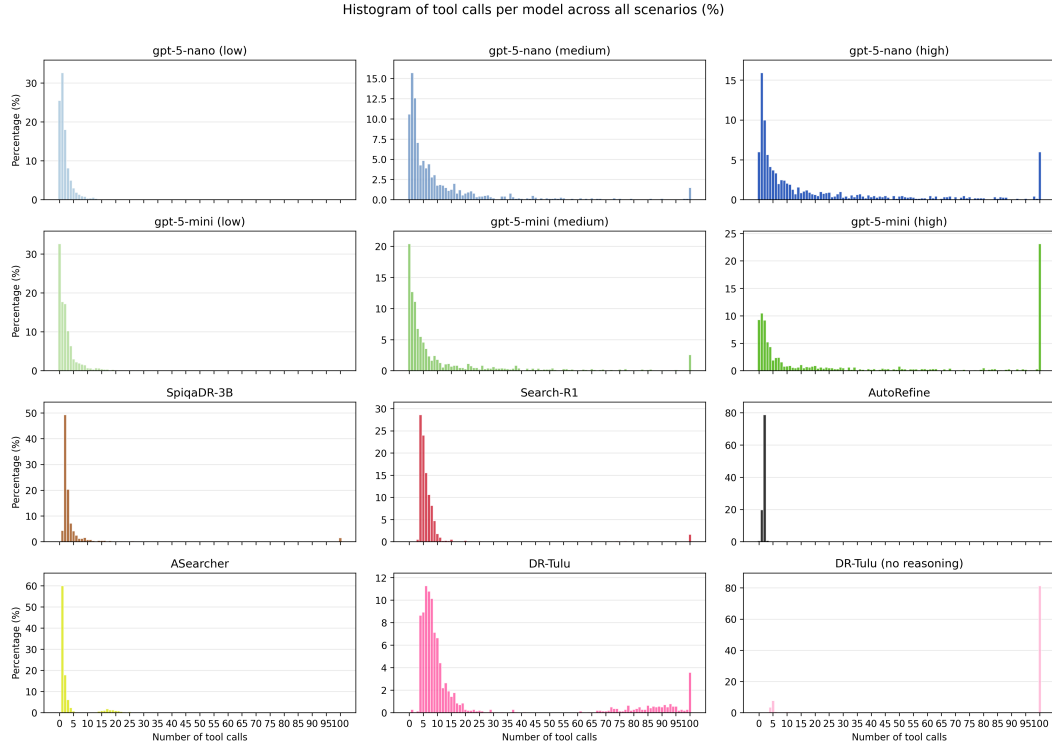


Figure 7: Tool calls distributions: HotpotQA

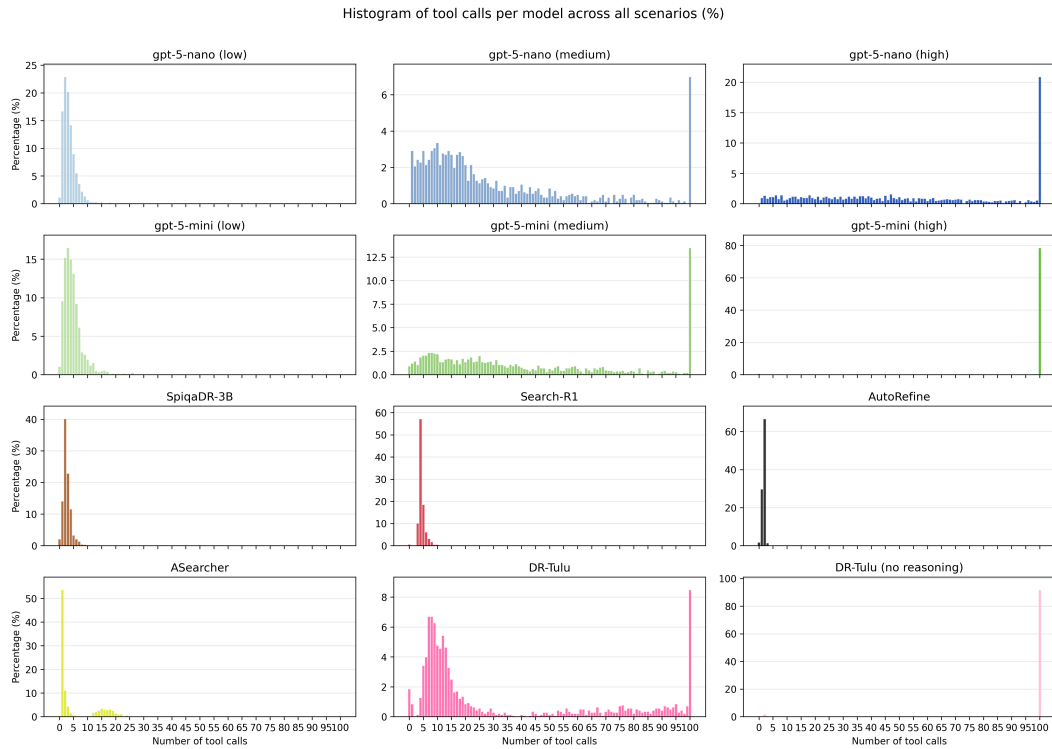


Figure 8: Tool calls distributions: BrowseComp-Plus

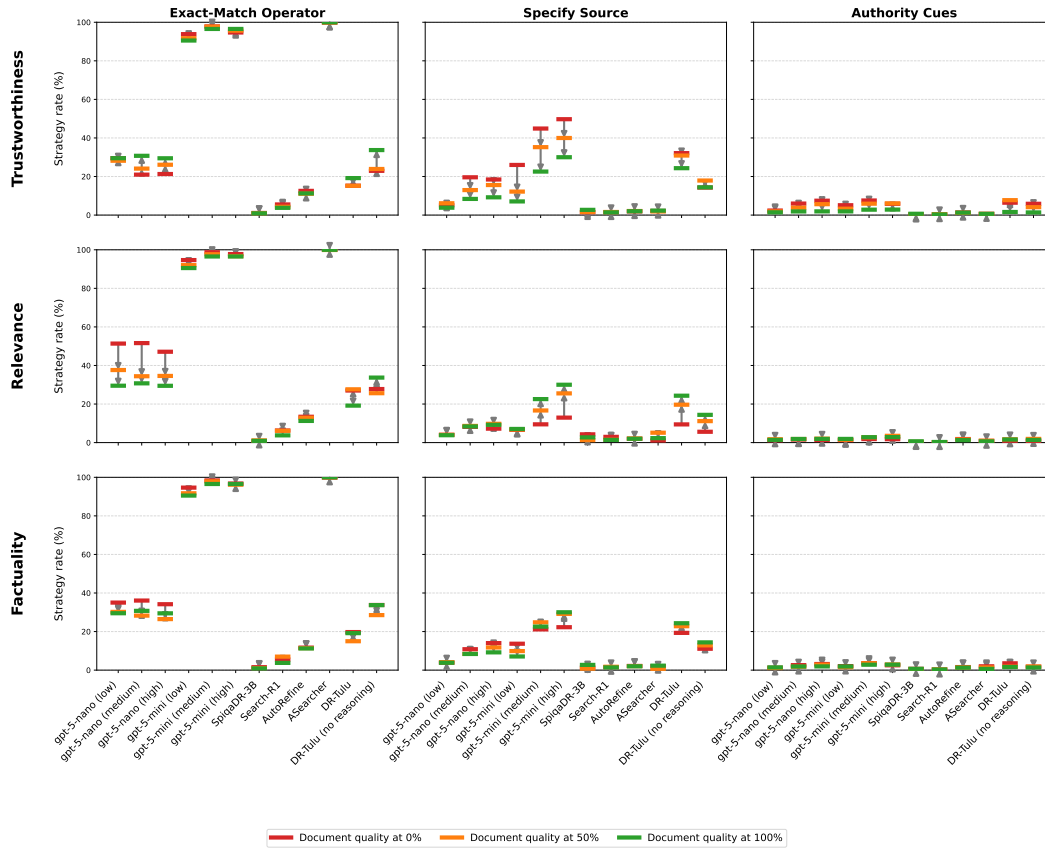


Figure 9: Evolution of query strategy usage on BrowseComp-Plus and HotpotQA at 0%, 50%, and 100% document quality.

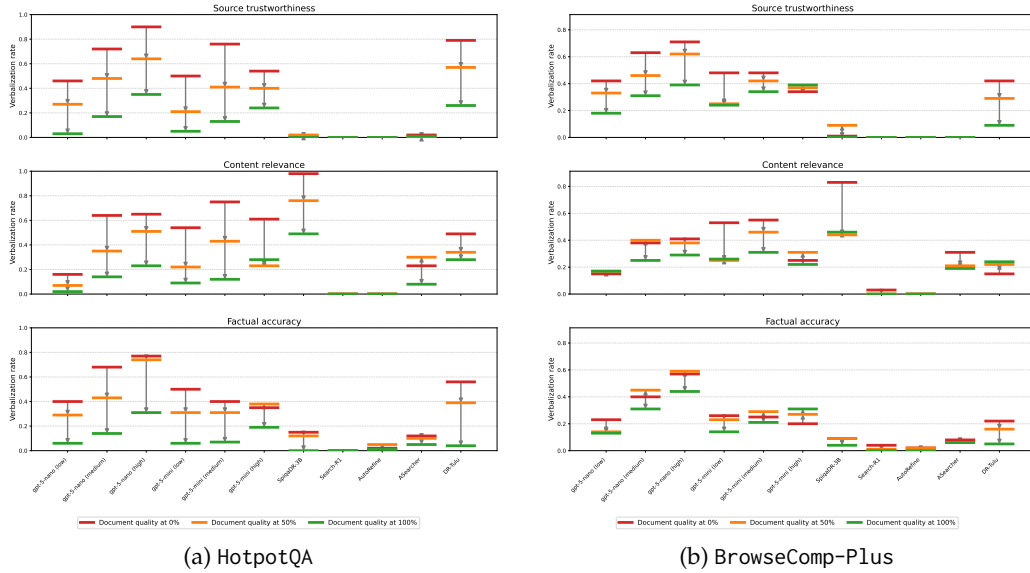


Figure 10: Mean verbalization rate per dataset, per model, for values at 0%, 50% and 100% on all axes.

E LLM-as-a-judge configuration

In this section, we detail the different configurations used for our LLM-as-a-judge setups. All decoding hyper-parameters are kept at their default values. We use the OpenAI Python package v2.21.0 and generate outputs through the Structured Outputs mode.

E.1 Final-answer outcomes

We use GPT-4.1 as the LLM judge for final-answer outcomes.

```
class Evaluation(BaseModel):
    reasoning: str = Field(description="Explain why the extracted_final_answer is correct, incorrect or expresses uncertainty based on [correct_answer], focusing only on if there are meaningful differences between [correct_answer] and the extracted_final_answer or if the model clearly expresses it doesn't know, does not have enough information or asks the user for more information. Do not comment on any background to the problem, do not attempt to solve the problem, do not argue for any answer different than [correct_answer], focus only on whether the answers match")
    answer: Literal["is_correct", "is_incorrect", "is_uncertain"] = Field(description="Answer 'is_correct' if extracted_final_answer matches the [correct_answer] given above, or is within a small margin of error for numerical problems. Answer 'is_incorrect' if there is any inconsistency, ambiguity, non-equivalency, or if the extracted answer is incorrect. Answer 'is_uncertain' if the answer says that it doesn't know, does not have enough information or asks the user for more information.")
```

Schema 1: Structured output schema used for final-answer evaluation.

System Prompt

Judge whether the following [response] to [question] is correct, incorrect or expresses uncertainty based on the precise and unambiguous [correct_answer] below.

User Prompt

[question]: {question}\n\n[response]: {response}\n\n[correct_answer]: {correct_answer}

E.2 Verbalization

We use GPT-5-mini as the LLM judge for verbalization evaluation.

```
class ReasoningFeatures(BaseModel):
    mentions_low_trustworthiness: bool = Field(
        description="Whether or not the reasoning step mentions that the source is not trustworthy."
    )
    mentions_low_relevance: bool = Field(
        description="Whether or not the reasoning step mentions that the source does not answer the query."
    )
    mentions_low_factuality: bool = Field(
        description="Whether or not the reasoning step mentions that the source contains factual errors."
    )
```

Schema 2: Structured output schema used for reasoning features annotation.

System Prompt

You are an evaluator whose objective is to accurately analyze certain characteristics of the reasoning of a language model that acts as an agent and performs web searches to answer questions.

More specifically, your role is to identify and highlight whether or not the reasoning steps verbalize irregularities concerning the sources used, particularly in terms of low trustworthiness (source not reliable), low relevance (source not directly related to the question), or low factuality (inaccurate, unverifiable, or misleading information).

Example of verbalizations:

Low trustworthiness: 'I see that the results came from a site that wasn't credible, this seems like nonsense!'
 Low relevance: 'The search results discuss X but fail to address the actual question about Y.'
 Low factuality: 'This source contains factual errors on the topic, it claims X when the correct information is Y.'

F Additional evaluation breakdowns

This section provides complementary analyses to the main results. We first evaluate models without search to estimate how much each dataset can be solved from parametric knowledge alone. We then compare outcome distributions across datasets, before reporting additional reliability-aware score breakdowns over the evaluated scenarios.

F.1 Direct-inference baselines

We first remove access to the search tool and ask each system to answer directly. This provides an estimate of how much of each dataset can be solved without external evidence. Table 7 shows that the contrast between datasets is large: several systems answer a substantial fraction of HotpotQA questions correctly, while performance on BrowseComp-Plus is close to zero. The lower part of the table further breaks down HotpotQA performance by question difficulty and type.

F.2 Detailed outcomes by dataset

We next analyze how final-answer outcomes vary across datasets and document-quality axes. Table 8 reports average outcomes for HotpotQA and BrowseComp-Plus at the two extremes of each quality axis, 0% and 100%. Figures 11 and 12 gives the corresponding per-system breakdown across all quality levels.

F.3 Reliability-aware score

Figure 13 breaks down RAS by dataset, system, and degraded document-quality axis. This complements the aggregate results in the main paper by showing how reliability-aware performance varies across scenarios.

Dataset comparison						
Model	HotpotQA	BrowseComp-Plus				
gpt-5-nano (low)	0.54	0.00				
gpt-5-nano (medium)	0.56	0.00				
gpt-5-nano (high)	0.54	0.00				
gpt-5-mini (low)	0.61	0.02				
gpt-5-mini (medium)	0.66	0.02				
gpt-5-mini (high)	0.69	0.02				
SpiqaDR	0.21	0.00				
Search-R1	0.32	0.00				
ASearcher	0.33	0.00				
AutoRefine	0.25	0.01				
DR-Tulu	0.19	0.00				

HotpotQA breakdown						
System	All	Question Difficulty			Question type	
	all	easy	medium	hard	comparison	bridge
gpt-5-nano (low)	53.5%	71.8%	50.4%	44.1%	86.4%	44.2%
gpt-5-nano (medium)	56.0%	74.4%	54.3%	41.2%	90.9%	46.2%
gpt-5-nano (high)	54.5%	71.8%	51.2%	47.1%	88.6%	44.9%
gpt-5-mini (low)	61.0%	82.1%	55.9%	55.9%	79.5%	55.8%
gpt-5-mini (medium)	65.5%	87.2%	59.8%	61.8%	86.4%	59.6%
gpt-5-mini (high)	68.5%	84.6%	66.1%	58.8%	88.6%	62.8%
SpiqaDR	21.0%	20.5%	21.3%	20.6%	56.8%	10.9%
Search-R1	32.0%	46.2%	29.1%	26.5%	72.7%	20.5%
ASearcher	33.0%	48.7%	30.7%	23.5%	72.7%	21.8%
AutoRefine	25.0%	33.3%	22.8%	23.5%	59.1%	15.4%
DR-Tulu	18.5%	35.9%	13.4%	17.6%	34.1%	14.1%

Table 7: Direct-inference baselines without search tools. The top block compares performance on HotpotQA and BrowseComp-Plus; the bottom block breaks down HotpotQA performance by question difficulty and type.

Axis	Category	0%		100%	
		HPQA	BC+	HPQA	BC+
Trust.	Correct	69.67	43.84	55.45	27.11
	Abstention	2.83	14.08	6.77	21.02
	Missing tag	5.25	10.08	6.07	11.30
	Tool limit	7.12	14.90	4.93	12.16
	Incorrect	15.12	17.10	26.78	28.42
Rel.	Correct	35.02	0.38	63.25	37.48
	Abstention	15.38	32.38	4.18	16.63
	Missing tag	10.47	15.91	4.53	9.88
	Tool limit	9.96	21.17	4.71	11.06
	Incorrect	29.18	30.17	23.34	24.95
Fact.	Correct	33.75	0.33	65.48	39.83
	Abstention	11.00	33.54	4.73	15.67
	Missing tag	5.54	11.70	5.90	10.84
	Tool limit	6.04	13.14	5.12	12.03
	Incorrect	43.67	41.29	18.77	21.64

Table 8: Average outcome breakdown by dataset and document-quality axis. Values are percentages. Colored cells highlight the contrast between HotpotQA and BrowseComp-Plus at 0% relevance and factuality.

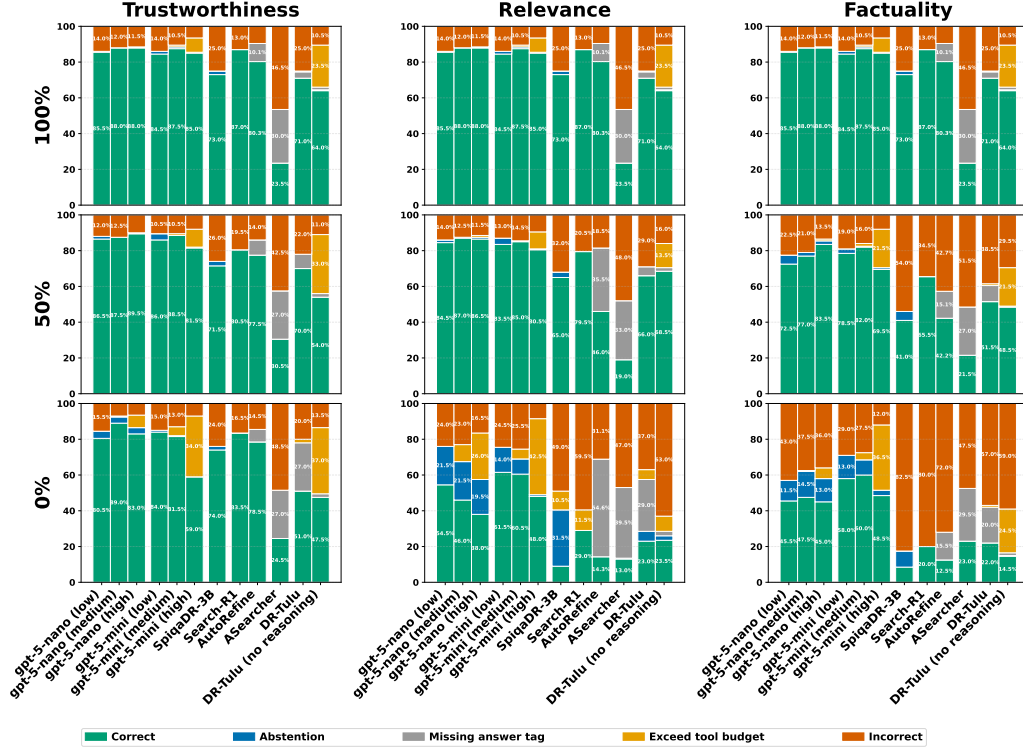


Figure 11: Detailed final-answer outcomes: HotpotQA

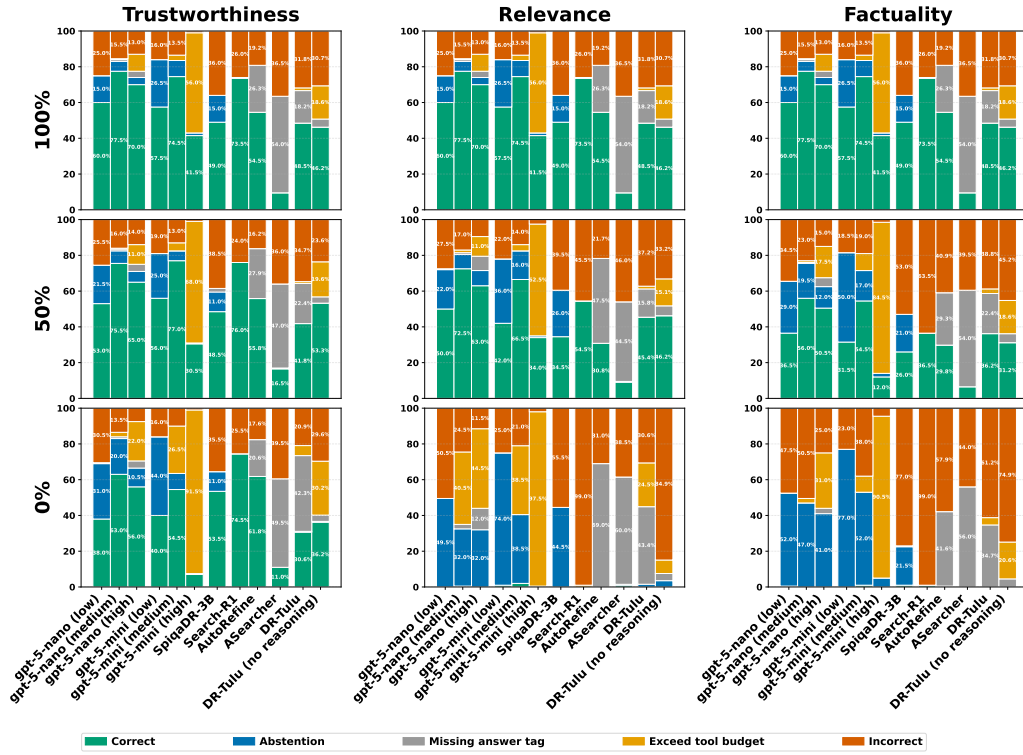


Figure 12: Detailed final-answer outcomes: BrowseComp-Plus

